

> head(iris)

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
1	5.1	3.5	(large) broad 1.4	0.2
2	4.9	3.0	1.4	0.2
3	4.7	3.2	(big) broad 1.3	0.2
4	4.6	3.1	1.5	0.4
5	5.4	3.6	"big" = 1.4	
6	5.0	3.4	1.7	0.3
		"medium" = 1.1		
		"small" = 1.0		

"big" = 1.4, "medium" = 1.1, "small" = 1.0



Ex:No: 8

Date: 16/04/26

Iris Flower Clustering Using K-Means Algorithm

Problem Description:

The Iris dataset contains 150 samples of three iris species (Setosa, Versicolor, Virginica) with four numerical features: Sepal Length, Sepal Width, Petal Length, Petal Width. The goal is to group the flowers into clusters based on these features using the K-Means clustering algorithm, without using the species labels.

Objective:

The objective of this program is to implement the K-Means clustering algorithm in R to group iris flowers based on their numerical attributes, visualize the clusters, and evaluate clustering performance against the actual species.

Procedure:

- Step 1: Install and Load Required Libraries

We install and load the necessary libraries for implementing K-Means clustering.

```
# Install and Load required libraries
```

```
install.packages("factoextra")
```

```
library(ggplot2)
```

```
library(cluster)
```

```
library(factoextra)
```

- Step 2: Load and Prepare the Dataset

We use the iris dataset (excluding the Species column) for clustering.

```
# Load the iris dataset
```

```
data(iris)
```

```
# Remove the Species column (only numeric features)
```

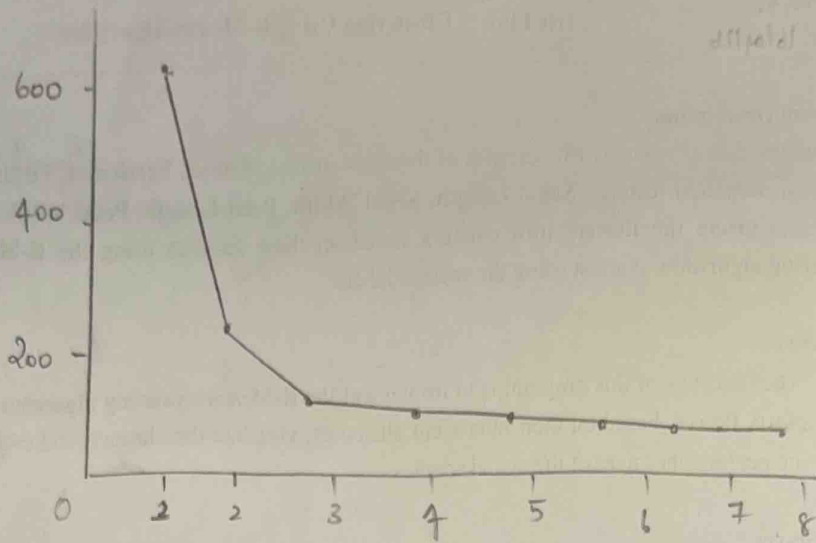
```
iris-data <- iris[, -5] # represent exception
```

```
# Display first few rows
```

```
head(iris-data)
```

Elbow Method for Optimal k

Total within sum of square

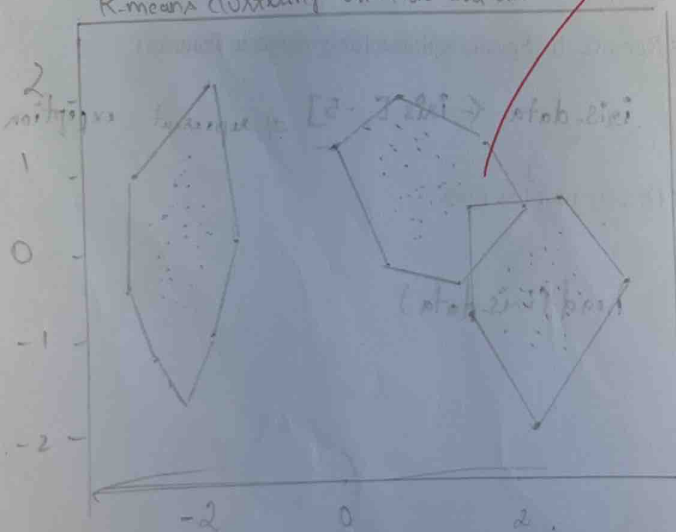


Number of clusters k

> print(kmeans\$centers)

	Sepal.Length	Sepal.width	Petal.width*length	Petal.width
1.	5.00600	3.428000	1.462000	0.246000
2.	5.901613	2.748387	4.393548	1.433871
3.	6.85000	3.073684	5.742105	2.071053

K-means clustering On Iris Dataset



- **Step 3: Determine the Optimal Number of Clusters (k)**

We use the Elbow Method to find the optimal number of clusters.

Elbow method to find optimal k using fviz_nbclust() ^{To find k}

```
fviz_nbclust(iris.data, kmeans, method = "wss") +
```

```
ggtitle ("Elbow Method for Optimal k")
```

Interpretation:

The optimal k is chosen where the within-cluster sum of squares (WSS) starts to level off (elbow point).

- **Step 4: Apply K-Means Clustering Algorithm**

We apply K-Means clustering with $k = 3$ (since iris dataset has 3 species).

```
# Apply K-Means clustering with 3 clusters
```

```
kmeans_model <- kmeans(iris.data = centers = 3, nstart = 25)
```

```
# Print cluster centers
```

```
print(kmeans_model$centers)
```

```
# Print cluster assignments
```

```
print(kmeans_model$cluster)
```

- **Step 5: Visualize Clusters**

We plot the clusters using Principal Component Analysis (PCA) for dimensionality reduction.

```
# Visualize clusters - fviz_cluster()
```

```
fviz_cluster(kmeans_model, data = iris.data, geom = "point",
```

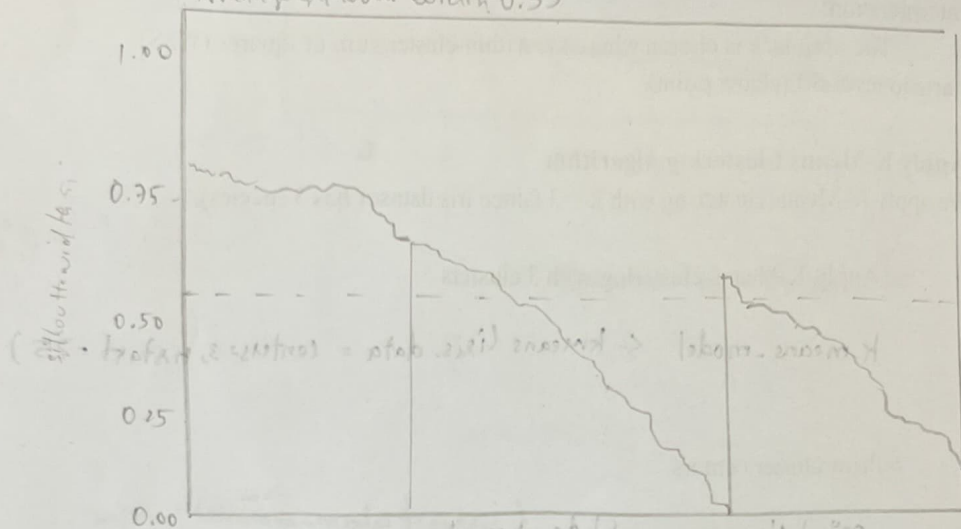
```
ellipse.type = "convex" +
```

```
ggtitle ("Kmeans clustering on iris Dataset").68
```

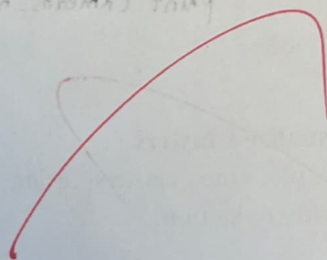
	cluster	size	ave. sil width
1)	1	50	0.80
2)	2	62	0.42
3)	3	38	0.45

cluster silhouette plot

Average silhouette width: 0.55



Print silhouette plot



- **Step 6: Evaluate Cluster Performance**

We calculate the Silhouette Score to measure clustering quality.

Compute silhouette score

```
sil_score <- silhouette(kmeans_model$cluster, dist(iris.data))
```

Plot silhouette analysis

```
fviz_silhouette(sil_score)
```

Result:

Thus, the K-Means clustering algorithm was successfully implemented in R, and the resulting clusters were visualized and evaluated, effectively revealing the natural groupings in the Iris dataset.

→ nchar() → return no. of character

→ gsub() → replace char

→ sub() → replace first char

→ Scan (What = character(), nmax = 3)
(Data-type) How many char to read

(plot) scatterplot, mainplot, ...
↓
ggplot

set.seed(123) → we use this because whenever we call the split there could be possible to change in train data.
set.seed(42)

(2000, 100) splitData = 100

geom_point → scatterplot

geom_smooth → line in LR

Completed
17/04/2020